Mutants overstate what LLM-written unit tests can find: reach on mutants and on real faults

Ook Lee

Department of Information Systems, Hanyang University, Seoul, Republic of Korea. ooklee@hanyang.ac.kr

Abstract

Context. Unit tests written by large language models (LLMs) are judged mostly by coverage and mutation scores, while buyers and maintainers need to know which faults the tests can find. *Objective.* We measure the reach of LLM-written tests, the share of faults they reveal, relative to developer tests, and ask whether mutation scores predict reach on real faults. *Method.* Three models (Claude Haiku 5.5, Claude Sonnet 5.5, DeepSeek V4.1 Flash) wrote pytest suites from source code alone, with up to eight independent attempts. We measured reach on 947 AST mutants of seven modules from four Python libraries and on 87 real faults from seven BugsInPy projects, with bootstrap intervals that resample modules or projects. *Results.* On mutants, a single attempt reaches 0.84–1.05 of what the existing tests kill and eight attempts exceed parity (1.15–1.20). On real faults, six attempts reveal only 0.10–0.15 of the faults, and all three models together 0.23. Most failing tests are wrong rather than revealing: 54–66% of attempts contain a test that fails on the faulty version, but only 4–5% reveal the fault. *Conclusions.* Mutation scores overstate the fault-finding reach of LLM-written tests by close to an order of magnitude in our setting; evaluations should include real faults and should separate revealing tests from failing tests.

Keywords Test generation · Large language models · Mutation testing · Real faults · Test oracle · BugsInPy

1 Introduction

Large language models now write unit tests in research tools and in industry [1, 2, 3]. Evaluations of such tests usually report coverage and mutation scores, and sometimes the share of bug reports from which a failure-reproducing test can be generated [4, 5]. A team that considers relying on generated tests needs a different quantity: of the faults its code contains, what share will the generated tests reveal, and how does that share compare with its own tests? We call this share the tool's reach. Reach differs from speed. A tool that writes many tests quickly but cannot express the right expectations reveals few faults however many tests it writes. Mutation scores are a natural proxy for reach, and mutant detection correlates with real-fault detection for developer and tool-generated Java suites [6]. It is not known whether the proxy holds for LLM-written tests, whose main weakness is the oracle: deciding what the correct output is [7]. An LLM that reads the code may encode the code's current behaviour as the expectation, which kills mutants that change that behaviour but cannot reveal a fault already present in the code.

We ask four research questions:

RQ1 How much of the mutant space that developer tests cover do LLM-written tests reach, and how does reach grow with the number of attempts?

RQ2 What share of real faults do LLM-written tests reveal, from source code alone?

RQ3 Why do the two measures differ? In particular, how often do LLM-written tests fail on faulty code without revealing the fault?

RQ4 What do the measured values imply for the decision to substitute generated tests for human testing?

Our contributions are a measurement of reach on both mutants and real faults with the same models and prompts, evidence that the two disagree by close to an order of magnitude, an account of the disagreement in terms of oracle errors, and a translation of the measured reach into a bound on the stakes at which substitution can pay. All code, prompts and results are public.

2 Background and related work

LLM-based test generation. TestPilot generated JavaScript unit tests with an LLM and reported statement coverage of 70.2% on 25 npm packages [1]. CodaMosa uses an LLM to escape coverage plateaus in search-based testing [2], and TestGen-LLM improves existing human-written tests at Meta under filters that require a measurable gain [3]. These works evaluate coverage, build and pass rates or acceptance by engineers, not the share of real faults revealed.

Reproducing reported bugs. LIBRO generates failure-reproducing tests from bug reports and reproduced 33% of 750 Defects4J cases [4]; SWT-Bench asks code agents to turn GitHub issues into tests that fail before and pass after the fix [5]. Both give the model a description of the fault. We give only the code, which is the situation of a team that wants tests to find faults it does not yet know about.

Mutants and real faults. Just et al. found that mutant detection correlates with real-fault detection, independently of coverage [6]. Our results suggest that for LLM-written tests the level differs greatly even if a correlation exists.

Reach in reliability models. Software reliability growth models distinguish the eventual number of detectable faults from the rate at which they are found [8]. Reach corresponds to the first quantity, and attempts play the role of testing time.

3 Study design

3.1 Models and prompts

We used Claude Haiku 5.5, Claude Sonnet 5.5 and DeepSeek V4.1 Flash through a commercial API gateway. For each target the model received only source code and was asked to write a thorough pytest file testing the public functions, including edge cases, with absolute imports and under 250 lines (prompts in the replication package). Each attempt is an independent call; reach after K attempts counts a mutant or fault as found if any of the first K attempts finds it.

3.2 Mutants (RQ1)

We generated AST mutants (arithmetic, comparison and Boolean operator replacement, constant replacement, negation removal) for eleven modules of five libraries (toolz, boltons, humanize, cachetools, more-itertools), capped at 150 per module. A mutant is killed by a suite if a test that passes on the original fails on the mutant. The human baseline is the project's existing test file for the module. Reach is the module-level ratio of mutants killed by the LLM tests to mutants killed by the existing tests, among mutants killed by either, averaged over the 7 modules with at least 30 mutants (947 mutants in boltons, humanize, more-itertools and toolz); intervals resample modules (1,500 resamples). LLM tests that failed on the original code were discarded before mutation analysis. Passing tests were identified from JUnit XML reports; an earlier version of the harness read test names from console output, which truncated names containing spaces and missed coloured output, and the affected kill vectors (humanize, and 5 of 24 attempts on boltons/strutils) were recomputed from the stored generated tests without new model calls.

3.3 Real faults (RQ2, RQ3)

We used BugsInPy [9]. For each bug we checked out the faulty and fixed commits, installed the project in its own environment and confirmed that the developer's fault-revealing test fails on the faulty and passes on the fixed version. Of the bugs in thirteen projects that we tried, {R['n_bugs']} in seven projects passed this check ({', '.join(f'{k} {v}' for k, v in sorted(PRJ.items(), key=lambda x: -x[1]))}); bugs in luigi, black and scrapy failed to build with their pinned dependencies. The model received the non-test source files changed by the fix (up to 40,000 characters each), in their faulty state, without the bug report or the fix. A fault is revealed if at least one generated test fails on the faulty version and passes on the fixed version. The

human baseline is the developer's fault-revealing test, so human reach is 1 by construction. Intervals resample projects and then bugs within projects (3,000 resamples).

3.4 Interpretation (RQ4)

In a Goel–Okumoto model [8] of testing, a technology that can find a pool of a faults at rate b and unit cost c gives a firm whose undetected faults cost f each a maximal net saving $fa\,h(\kappa)$, where $\kappa = abf/c$ and $h(x) = 1 - (1 + \ln x)/x$. A tool that reaches a share A of the human pool saves at most fAa. Whatever its price and speed, it cannot beat human testing for a firm whose stakes index κ exceeds $\bar{\kappa}(A)$, the solution of $h(\bar{\kappa}) = A$. We report $\bar{\kappa}$ at the measured reach; it is the upper end of the range of stakes in which substitution can pay.

4 Results

4.1 RQ1: reach on mutants

Table 1 RQ1: reach on mutants relative to existing tests (7 modules, 95% intervals over modules). Values above 1 mean the LLM tests kill more mutants than the existing tests.

Model	K = 1	K = 2	K = 4	K = 8
Claude Haiku 5.5	1.02 (0.89–1.18)	1.10 (0.94–1.28)	1.17 (0.96–1.39)	1.19 (0.99–1.40)
Claude Sonnet 5.5	1.05 (0.92–1.20)	1.13 (0.98–1.31)	1.17 (1.00–1.36)	1.20 (1.00–1.41)
DeepSeek V4.1 Flash	0.84 (0.69–1.04)	1.00 (0.83–1.19)	1.08 (0.91–1.28)	1.15 (0.96–1.37)

A single attempt reaches 0.84 to 1.05 of what the existing tests kill (Table 1, Fig. 1a). Reach rises with attempts; the Claude models are at parity after one attempt and DeepSeek after two, and all three exceed it at K = 8 (1.15–1.20). The intervals are wide because seven modules differ greatly: the existing tests kill from 0.47 to 0.93 of the mutants. The LLM tests are partly complementary to the existing tests: among mutants killed by either side, 17–18% are killed only by the LLM tests (K = 8). By the mutation measure, LLM-written tests are a substitute for the existing tests after one or two attempts.

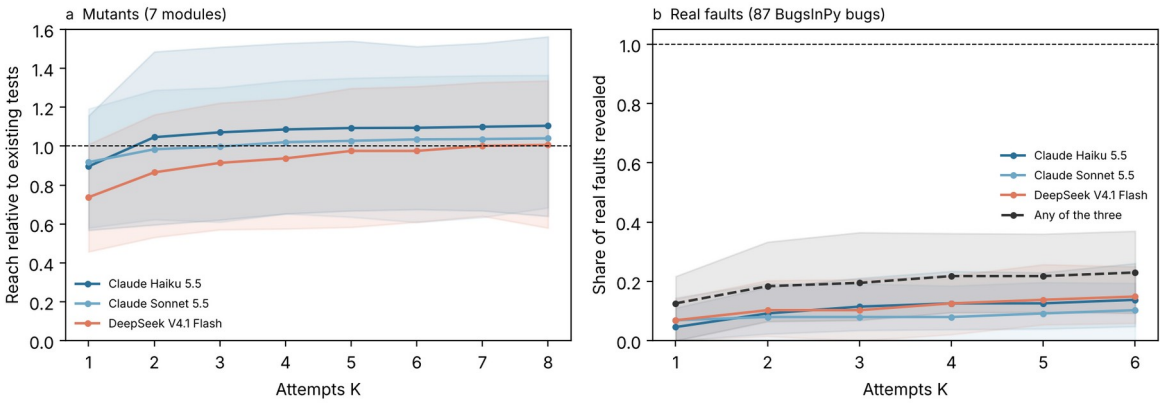


Fig. 1 Reach as a function of the number of attempts K. **a** Mutants, relative to the existing tests (dashed line: parity). **b** Real faults: share of 87 BugsInPy faults revealed, for each model and for the three together. Bands are 95% bootstrap intervals.

4.2 RQ2: reach on real faults

Table 2 RQ2: share of 87 real faults revealed (95% intervals resampling projects, then bugs). The developer's fault-revealing test reveals every fault by construction.

Model	K = 1	K = 3	K = 6
Claude Haiku 5.5	0.05 (0.00–0.11)	0.12 (0.04–0.21)	0.14 (0.05–0.26)
Claude Sonnet 5.5	0.07 (0.00–0.14)	0.08 (0.00–0.19)	0.10 (0.00–0.19)
DeepSeek V4.1 Flash	0.07 (0.00–0.14)	0.10 (0.00–0.21)	0.15 (0.06–0.25)
Any of the three	0.13 (0.00–0.22)	0.20 (0.07–0.36)	0.23 (0.09–0.37)

On real faults the picture reverses. A single attempt reveals 0.05–0.07 of the faults, six attempts 0.10–0.15, and the union of all three models' 18 attempts 0.23 (Table 2, Fig. 1b). Reach differs by project: with the three models together it is 0.43 on tqdm (7 faults), 0.31 on thefuck (32), 0.20 on httpie (5) and 0.16 on youtube-dl (38), and zero on the five faults of fastapi, tornado and cookiecutter. The ordering of models also changes: Claude Sonnet 5.5, the strongest on mutants at one attempt, is the weakest on real faults at six.

4.3 RQ3: failing tests are mostly wrong tests

Table 3 RQ3: outcomes of single attempts on real faults (87 faults × 6 attempts per model).

Model	Attempts	No runnable test	Some test fails on faulty version	Reveals the fault
Claude Haiku 5.5	522	1.0%	54.2%	4.4%
Claude Sonnet 5.5	522	0.0%	61.3%	5.2%
DeepSeek V4.1 Flash	522	1.1%	65.5%	5.0%

Almost every attempt produced runnable tests, and in 54–66% of attempts at least one test failed on the faulty version (Table 3). Only 4.4–5.2% of attempts revealed the fault, that is, contained a

test that also passed after the fix. The rest failed for other reasons: expectations that do not match the intended behaviour, environment assumptions or wrong use of the API. This explains the gap with RQ1. Mutation analysis discards tests that fail on the original code and counts a kill whenever a passing test fails on a mutant, so it rewards tests that pin down current behaviour. A real fault is already part of the current behaviour, and a test that pins it down passes on the faulty version. To reveal it, a test must encode the intended behaviour, which the code alone does not show: this is the oracle problem [7] in its plainest form. A developer who receives a failing generated test therefore cannot treat the failure as evidence of a fault without checking the test.

4.4 RQ4: implications for substitution

With six attempts, the measured real-fault reach implies $\bar{\kappa}$ of 1.7 to 2.0: a team whose stakes index exceeds about two cannot gain by replacing its fault-revealing tests with these generated tests at any price, whereas by the mutation measure (reach near 1) the bound disappears. The stakes index is the product of the human fault pool, the human detection rate and the cost of one escaped fault, divided by the cost of a unit of testing time, so a value of two is reached by most teams for which escaped faults are costly. The interpretation assumes that the developer's tests reveal all faults, which overstates the human side; it also treats generated tests as a substitute, not a complement. Generated tests can still be useful as a complement: they kill mutants the existing tests miss (RQ1).

5 Discussion

For practitioners. Generated tests that pass on the current code document its behaviour; they protect against regressions, which is what mutation scores measure. They are much weaker at finding faults that are already present. A failing generated test is more likely to be wrong than to reveal a fault; a review step is needed before acting on it.

For researchers. Mutation scores and coverage can make LLM-written tests look like substitutes for developer tests when, on real faults, they are not. Evaluations should report reach on real faults, separate revealing from failing tests, and report how reach grows with attempts. Benchmarks that give the model the bug report [4, 5] measure reproduction, a different task.

6 Threats to validity

Construct validity. Mutation reach is relative to the existing tests and real-fault reach is relative to the developer's fault-revealing test; the two are not on the same scale, and the human baseline

for real faults is 1 by construction, which favours the human side. *Internal validity.* We gave the model the files changed by the fix, which tells it where the fault is; reach without this hint would be lower. Models may have seen the projects, their tests or their fixes during training, which would raise reach; our values are therefore likely upper bounds. *External validity.* The real faults come from seven Python projects, 38 from youtube-dl and 32 from thefuck, so project-cluster intervals are wide; three projects failed to build; we did not test Java (Defects4J) or other languages. The mutants come from seven modules of utility libraries. *Conclusion validity.* With 87 faults, differences between models are within the intervals.

7 Conclusion

LLM-written unit tests reach or exceed parity with existing tests on mutants within one or two attempts but reveal about one real fault in seven, because most of their failures come from wrong expectations. Mutation scores overstate their fault-finding reach, and evaluations of generated tests should measure reach on real faults.

Data availability

Code, prompts, per-bug and per-module results and analysis scripts are available at https://doi.org/10.5281/zenodo.23652337 [to be updated with this version]. BugsInPy and the studied libraries are public.

Declarations

Funding Not applicable. *Competing interests* The author declares no competing interests. *Ethics* No human participants were involved. *Author contributions* O.L. conceived the study, built the pipeline, ran the analyses and wrote the manuscript. *Use of generative AI* The studied tests were written by the three LLMs named above. An AI coding assistant (Claude Code) was used to write and run the analysis code and to draft and edit the manuscript text; the author reviewed the results and takes responsibility for the content.

References

1. Schäfer M, Nadi S, Eghbali A, Tip F (2024) An empirical evaluation of using large language models for automated unit test generation. IEEE Trans Softw Eng 50(1):85–105
2. Lemieux C, Inala JP, Lahiri SK, Sen S (2023) CodaMosa: escaping coverage plateaus in test generation with pre-trained large language models. In: Proc 45th Int Conf Software Engineering (ICSE), pp 919–931. https://doi.org/10.1109/ICSE48619.2023.00085
3. Alshahwan N, Chheda J, Finogenova A, Gokkaya B, Harman M, Harper I, Marginean A, Sengupta S, Wang E (2024) Automated unit test improvement using large language models at Meta. In: Companion Proc 32nd ACM Int Conf Foundations of Software Engineering (FSE '24)
4. Kang S, Yoon J, Yoo S (2023) Large language models are few-shot testers: exploring LLM-based general bug reproduction. In: Proc 45th Int Conf Software Engineering (ICSE), pp 2312–2323
5. Mündler N, Müller MN, He J, Vechev M (2024) SWT-Bench: testing and validating real-world bug-fixes with code agents. In: Advances in Neural Information Processing Systems 37. https://doi.org/10.52202/079017-2601
6. Just R, Jalali D, Inozemtseva L, Ernst MD, Holmes R, Fraser G (2014) Are mutants a valid substitute for real faults in software testing? In: Proc 22nd ACM SIGSOFT Int Symp Foundations of Software Engineering (FSE), pp 654–665
7. Barr ET, Harman M, McMinn P, Shahbaz M, Yoo S (2015) The oracle problem in software testing: a survey. IEEE Trans Softw Eng 41(5):507–525. https://doi.org/10.1109/TSE.2014.2372785
8. Goel AL, Okumoto K (1979) Time-dependent error-detection rate model for software reliability and other performance measures. IEEE Trans Reliab 28(3):206–211
9. Widyasari R et al (2020) BugsInPy: a database of existing bugs in Python programs to enable controlled testing and debugging studies. In: Proc 28th ACM ESEC/FSE, pp 1556–1560. https://doi.org/10.1145/3368089.3417943